설비 고장을 예측하는 AI 데이터 분석

모듈·표준 라이브러리·파일 읽기

파일 입출력

posco

CONTENTS

파일 입출력

PART 01

파일 입출력

txt 다루기 · csv 읽기 · csv 쓰기

학습 흐름

① 파일을 여는 도구 **open()** 과 모드(r·w·a) · 인코딩
② **with open** · 자동 닫힘 · txt 읽고 쓰기
③ **csv.reader / csv.writer** 로 표 데이터 다루기

이 과정의 실데이터 — 소성가공 프레스

공정 — 소성가공 프레스

금속을 강한 힘으로 눌러 형태를 만드는 설비. 작동 중 진동과 전류를 기록한다.

정상과 이상

정상이면 진동·전류가 일정 범위. 금형 마모나 이물이 끼면 값이 튀고 상태가 1로 기록된다.

다룰 파일

data/08_press.csv — 설비ID·시각·진동X·진동Y·전류·상태. 파일 읽기·쓰기 실습의 대상 실데이터.

PART 01 · 파일 입출력

데이터를 파일로 주고받는 이유

> **데이터는 파일에, 코드는 따로 — 분석은 파일을 여는 것에서 시작**

앞서 파일을 찾는 법을 배웠고, 이제 그 파일을 열어 안을 들여다본다.

설비 데이터는 하루에도 **수만 줄**. 코드에 직접 적을 수 없다.

데이터는 **파일**에 저장하고, 코드는 그 파일을 열어 읽는다 — **데이터와 코드의 분리**.

PART 01 · 파일 입출력

파일로 주고받는 두 가지 장점

> **데이터가 바뀌어도 코드는 그대로 · 결과는 파일로 저장해 공유·재사용**

01

코드 고정

데이터가 바뀌어도 코드는 그대로.

오늘 파일이든 어제 파일이든 **같은 코드로 분석**.

02

공유

만든 결과를 **파일로 저장해 전달·재사용**.

다음 작업에 이어 쓰기 좋다.

posco

open() 함수 구조

> **open(파일명, 모드, 인코딩) — 내장 함수라 import 없이 바로 사용**

```
open('data/08_press.csv', 'r', encoding='utf-8')
```

파일명 · 모드 · 인코딩

open(파일명, 모드, 인코딩) 으로 파일을 연다.

open 은 **내장 함수** — import 가 필요 없다.

결과로 받는 것은 파일이 아닌, 파일을 다룰 **연결 통로**(파일 객체).

open() — 코드로 확인

파일명·모드·인코딩으로 파일 열기

```
f = open('sensor.txt', 'r', encoding='utf-8')   # f = 파일 객체(연결 통로)
print(type(f).__name__)   # TextIOWrapper
f.close()
```

posco

인코딩(utf-8)이 중요한 이유

> **인코딩은 글자를 숫자로 바꾸는 약속 — 한글 깨지면 utf-8 ↔ cp949 전환**

파일을 열었는데 **한글이 깨진** 경험 — 원인은 인코딩.

컴퓨터는 글자를 **숫자로 바꿔 저장**한다. 그 약속이 인코딩이고, **저장과 읽기의 약속이 다르면 깨진다**.

오늘날 표준은 **utf-8**. 다만 윈도우 엑셀로 저장한 CSV는 **cp949**일 수 있다.

한글이 깨지면 utf-8 ↔ cp949 를 바꿔 본다.

> **utf-8**
> 오늘의 표준 — 거의 모든 환경에서 안전

> **cp949**
> 윈도우 엑셀 CSV — 깨지면 여기로 전환

posco

파일 모드 r · w · a 차이

> **모드는 파일을 어떤 목적으로 열지 정한다 — w 와 a 헷갈리면 데이터가 날아간다**

모드	의미	동작
`r`	**읽기**	내용 읽기 (기본값)
`w`	**새로 쓰기**	기존 삭제 후 작성
`a`	**이어 쓰기**	기존 뒤에 추가

⚠ 새로 만들 때만 w, 이어 붙일 때는 a

posco

read · readline · readlines 차이

> **내용을 꺼내는 세 가지 방법 — 끝의 s 한 글자가 결과를 바꾼다**

방법	읽는 범위	돌려주는 것
`read`	전체	한 덩어리 **문자열**
`readline`	한 줄	한 줄 **문자열**
`readlines`	전체	줄 단위 **리스트**

줄 단위 센서 로그에는 readlines 또는 파일 직접 반복이 잘 어울린다

read 3종 — 코드로 확인

> readlines로 모든 줄을 리스트로 읽기

```
with open('data/08_press.csv', 'r', encoding='utf-8') as f:
    lines = f.readlines()
print(type(lines).__name__, len(lines))   # list 13
```

PRACTICE · 실습

실습 1. open으로 파일 읽기

open으로 파일을 열어 read·readlines로 내용을 읽기

단계

① open으로 파일을 읽기 모드 r, utf-8로 열기
② read로 전체를 한 문자열로 읽어 출력
③ readlines로 줄 리스트로 읽어 출력
④ 두 방식의 결과 차이를 비교하고 파일을 close

예상 결과 read는 한 덩어리 문자열 / readlines는 줄 리스트

실습 1. open으로 파일 읽기 정답

```
f = open("data/08_press.csv", "r", encoding="utf-8")
text = f.read()          # 전체를 한 문자열로
print(text)
f.close()
f = open("data/08_press.csv", "r", encoding="utf-8")
lines = f.readlines()    # 줄 리스트로
print(lines)
f.close()
```

파일을 닫아야 하는 이유

> **열었으면 닫는다 — 자원 반납과 안전한 저장의 안전장치**

파일을 열었으면 반드시 닫는다 — 이유는 두 가지.

① **자원 반납** — 안 닫으면 쌓여, 나중에 파일을 못 열 수 있다.

② **저장 보장** — 쓰기에서 닫지 않으면 임시 공간의 내용이 **실제 파일에 기록되지 못하고 사라진다**.

닫기는 안전장치. 하지만 깜빡하기 쉬워, **with 가 이를 자동으로 해결**한다.

with open() 컨텍스트 매니저

> **블록이 끝나면 파일이 자동으로 닫힘 — 실무의 기본 형태**

```
with open(...) as f:
```

with open(...) as f: 블록 안에서 파일을 다룬다.

블록이 끝나면 파일이 **자동으로 닫힌다** — close 불필요.

오류가 나도 **안전하게 닫힌다**. 그래서 실무의 기본 형태.

posco

with open — 코드로 확인

> 블록을 벗어나면 자동으로 닫힘

```
with open('data/08_press.csv', 'r', encoding='utf-8') as f:
    for line in f:
        print(line.strip())   # 블록 밖으로 나가면 자동 닫힘
```

PRACTICE · 실습

실습 2. with open으로 파일에 쓰기

with open의 w 모드로 파일에 내용을 쓰고 다시 읽어 확인하기

단계

① with open으로 파일을 쓰기 모드 w, utf-8로 열기
② write로 내용을 쓰기(줄을 나눌 땐 줄바꿈 기호)
③ with 블록이 끝나면 파일이 자동으로 닫힘
④ r 모드로 다시 열어 쓴 내용을 확인

예상 결과 쓴 내용이 그대로 읽힘

실습 2. with open으로 파일에 쓰기 정답

```
with open("08_out.txt", "w", encoding="utf-8") as f:
    f.write("프레스 점검 시작\n")
    f.write("이상 없음\n")
with open("08_out.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

PRACTICE · 실습

실습 3. a 모드로 기록 이어붙이기

a 모드로 기존 내용을 지우지 않고 새 기록을 이어 붙이기

단계

① with open으로 파일을 추가 모드 a로 열기
② write로 새 기록 문장을 쓰기
③ w 모드와 달리 기존 내용이 보존됨을 확인
④ r 모드로 열어 전체가 쌓였는지 확인

예상 결과 기존 내용 + 새 기록이 함께 남음

실습 3. a 모드로 기록 이어붙이기 정답

```
with open("08_log.txt", "a", encoding="utf-8") as f:
    f.write("추가 기록\n")   # a = 기존 내용 뒤에 이어 붙임
with open("08_log.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

posco

csv가 제조 데이터의 기본 형식인 이유

> **csv는 쉼표로 구분된 표 형식 텍스트 — 가볍고 호환성 높아 제조 데이터의 표준**

설비·제조 현장 데이터는 대부분 **csv** — 데이터를 쉼표로 구분해 저장하는 단순한 텍스트.

표를 텍스트로 옮긴 형태. 행은 줄바꿈, 열은 쉼표로 나뉜다.

표준이 된 이유는 **가볍고 · 호환성 높고 · 단순하다** 는 세 가지. 엑셀·파이썬·DB 어디서나 읽고 쓴다.

앞으로 다룰 데이터도 거의 다 csv.

posco

CSV 구조 — 행·열·구분자·헤더

> **읽기 전에 첫 줄이 헤더인지 확인하는 습관이 필요**

요소	역할	예시
헤더	열 제목	`timestamp,vibration`
행	한 건 기록	`09:00,PUMP-01,0.32`
열	값 종류	시각 / 설비 / 진동
구분자	열 구분	쉼표 (,)

csv.reader 구조

> **각 행은 리스트로, 모든 값은 문자열로 반환 — 계산하려면 형변환 필요**

```
import csv → reader = csv.reader(f)
```

import csv 후 **csv.reader** 로 읽는다.

with open 으로 연 파일을 reader 에 넘기고, **반복문으로 한 행씩** 받는다.

각 행은 **리스트**, 모든 값은 **문자열** — 쉼표로 자르는 일은 reader 가 알아서 한다.

csv.reader — 코드로 확인

한 행씩 리스트로 읽기

```
import csv
with open('data/08_press.csv', 'r', encoding='utf-8') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)   # 각 행이 리스트로 출력됨
```

읽은 데이터를 리스트로 다루기

> **숫자 계산 전에는 반드시 float·int 로 바꾸는 습관**

01 · INDEX

인덱스 추출

행은 **리스트**. **0부터** 시작하는 번호로 열을 고른다 — 세 번째 열은 인덱스 **2**.

02 · CAST

형변환

문자열 값은 **float** 로 바꿔야 계산된다. 빠뜨리면 문자열끼리 비교해 오류.

PRACTICE · 실습

실습 4. csv.reader로 CSV 읽기

csv.reader로 CSV 파일을 행 단위로 읽기

단계

① csv 모듈을 import
② with open으로 CSV를 읽기 모드 utf-8로 열기
③ csv.reader로 reader 객체를 만들기
④ for로 각 행(리스트)을 하나씩 꺼내 출력

예상 결과 각 행이 리스트로 출력

실습 4. csv.reader로 CSV 읽기 정답

```
import csv
with open("data/08_press.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    next(reader)                # 헤더 건너뛰기
    for row in reader:
        print(row[0], row[5])   # 설비ID, 상태
```

posco

csv.writer 구조

> **csv.writer 는 리스트를 행으로 써 주는 reader 의 정확히 반대**

```
writer = csv.writer(f) → writer.writerow([...])
```

csv.writer 로 리스트를 한 행으로 저장한다.

with open 을 w 모드로 열되 **newline=''** 옵션을 더한다.

writerow(리스트) 로 한 행씩 — reader 와 정확히 반대 방향.

posco

csv.writer — 코드로 확인

리스트를 한 행으로 기록 (newline 필수)

```
import csv
with open('result.csv', 'w', encoding='utf-8', newline='') as f:
    writer = csv.writer(f)
    writer.writerow(['시각', '설비'])
    writer.writerow(['09:00', 'PUMP-01'])
```

행 단위 준비와 newline 옵션

> **CSV 쓰기에는 항상 newline='' — 패턴으로 기억**

01 · LIST

행 단위 준비

한 행의 값들을 **리스트로 묶는다**. 모든 행의 열 개수를 맞춰야 표가 어긋나지 않는다.

02 · NEWLINE

newline 옵션

쓰기 open 에 **빈 따옴표** 를 줘 윈도우에서 빈 줄이 끼는 것을 막는다.

PRACTICE · 실습

실습 5. csv.writer로 CSV 쓰기

csv.writer로 데이터를 CSV 파일에 행 단위로 쓰기

단계

① csv를 import
② with open으로 w·utf-8·newline 옵션으로 열기
③ csv.writer로 writer 객체를 만들기
④ writerow로 헤더와 각 데이터 행을 쓰기

예상 결과 새 CSV에 헤더+데이터 행이 저장됨

posco

실습 5. csv.writer로 CSV 쓰기 정답

```
import csv
with open("08_summary.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["설비ID", "상태"])
    writer.writerow(["PRESS-10", 1])
    writer.writerow(["PRESS-11", 1])
```

posco

csv.DictReader 맛보기

> **번호 대신 이름으로 — 열 순서가 바뀌어도 안전**

번호 방식

csv.reader

인덱스 번호 로 값 선택 — 세 번째 열이면 인덱스 2.

이름 방식

DictReader

헤더 이름 으로 값 선택 — 열 순서가 바뀌어도 안전.

posco

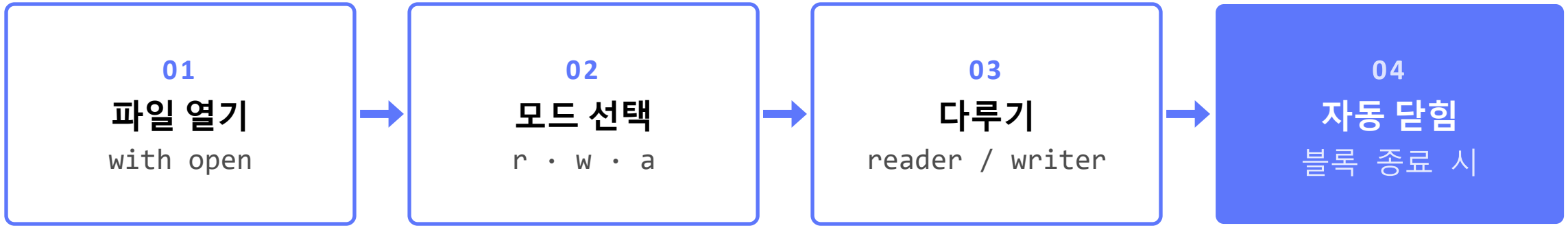
csv 다룰 때 흔한 오류

> **네 가지는 누구나 겪는다 — 막혔을 때 이 목록을 떠올리면 빠르게 해결**

증상	원인	해결
한글 깨짐	인코딩 불일치	utf-8 / cp949 바꿔 보기
빈 줄 끼임	newline 누락	쓰기 open 에 newline=''
계산 오류	문자열 상태	float · int 변환
첫 줄 오류	헤더 포함	헤더 건너뛰기

파일 입출력 전체 흐름

> **with open 한 줄이면 열기·다루기·자동 닫힘이 자연스럽게 이어진다**



이 네 단계가 데이터를 **불러오고 저장하는 뼈대**

파일 입출력 — 한 문장 요약

> **with open 으로 열고 reader 로 읽어 형변환 — writer 로 리스트를 행으로 저장**

with open 으로 안전하게 열고, 읽기는 **reader** 로 리스트를 받아 **형변환**, 쓰기는 **writer** 로 리스트를 행으로 저장한다.

앞서 import · os 로 파일을 찾았다면, 이번 시간는 그 파일을 열고 · 읽고 · 처리하고 · 저장 한다.

둘을 합치면 **데이터 처리의 완전한 한 사이클** 이 완성된다.

이후 배울 Pandas 도 이 흐름 위에 있다.

PRACTICE · 실습

실습 6. CSV 읽어 조건 저장하기

CSV를 읽어 조건에 맞는 데이터만 골라 새 CSV로 저장하기

단계

① csv를 import
② csv.reader로 읽고 첫 줄 헤더는 건너뛰기
③ 값을 float로 변환해 기준(90) 초과 행만 리스트에 모으기
④ csv.writer로 모은 행들을 새 CSV에 저장

예상 결과 기준 초과 센서만 담긴 새 CSV

실습 6. CSV 읽어 조건 저장하기 정답

```
import csv
fails = []
with open("data/08_press.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    header = next(reader)               # 헤더 보관
    for row in reader:
        if row[5] == "1":               # 상태(5열)==1
            fails.append(row)
with open("08_fail.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(header)
    for row in fails:
        writer.writerow(row)
```